
Improving ISO 20022 Settlement in International Trade Through Decentralized Escrow and Identity Layers

Colabonate Stack, ICP Canister, Nostr, Lightning, and XID

Author: Deniz Yilmaz, Colabonate

Status: Working draft for committee proposal

Date: August 6, 2026

Addressee: [Name of the target committee]

Target repository: [Target repository: to be filled in at integration]

Open item: The name of the addressed committee and the target repository are not known to the author and are therefore marked as placeholders. Both fields can only be finalized once the paper is integrated into the target repository.

Table of Contents

- 0. Executive Summary
 - 0.1 Executive Summary for a Broad Committee (Non-Technical Audience)
 - 0.2 In-Depth Summary for the Technical Audience
- 1. Background and Objective
- 2. ISO 20022: Status Quo and Structural Gaps
- 3. The Colabonate Stack: An Overview
- 4. Escrow Strategy via ICP Canister
- 5. Identity Layer: Nostr, XID, and the Trust Model
- 6. Lightning as the Payment Layer
- 7. Comparative Overview for the Committee
- 8. Proof-of-Concept Sketch and ISO 20022 Adapter Blueprint
- 9. Conclusion and Recommendation to the Committee
- References

0. Executive Summary

This summary is provided in two versions: Section 0.1 as an accessible summary for a broad committee without an extensive payments/technical background, and Section 0.2 as an in-depth version for the technical audience.

0.1 Executive Summary for a Broad Committee (Non-Technical Audience)

International payments are currently undergoing a mandatory transition to the new ISO 20022 messaging standard. This standard, however, only governs how payment information is described and transmitted in a message — it does not answer three other questions that are at least as important for the security of an international trade deal: How do you establish, in a tamper-proof way, who the other party to the contract really is? What happens if the two sides cannot agree on whether a deal was fulfilled? And: who holds the money until both sides have done their part — and what stops that party from simply keeping it? These three questions (identity, arbitration, escrow) remain unanswered after the ISO 20022 transition, because a pure messaging standard simply cannot govern them.

The central thesis of this paper: the Colabonate stack does not replace ISO 20022, but **complements** the standard with exactly these three missing answers. An escrow mechanism implemented in code — a so-called ICP canister, technically secured via native Bitcoin — holds the money, so that no party, not even Colabonate itself, can ever access it; a key-based identity layer — Nostr, a decentralized protocol for digital identity, complemented by the proposed XID component — makes trading partners unique and verifiable; a multi-stage, human-led arbitration process resolves disputes. The technical core idea rests on two principles. First: every release of the escrowed funds is bound to a digital signature that is valid exclusively for that one specific deal. Second: in a dispute, humans decide who is right — the system then only carries out that decision technically, without the arbitration body itself ever gaining access to the money. This resolves a classic either/or: either you entrust the money to a third party — risking that it keeps the funds — or you forgo any possibility of arbitration. Here, both hold at once: irreversibly secure settlement *and* a binding arbitration process. All three additions operate on top of ISO 20022 — the standard itself remains untouched and is, through this added protection, even strengthened.

This paper consistently and honestly distinguishes between what has already been demonstrated and what is still a draft or a proposal. **Already demonstrated:** in a controlled test environment, a complete trade with staged fund release (in three steps:

25%, 50%, 25%) has already been successfully carried out [Canister-README] — the escrow mechanism provably works. **Adopted as a design decision, but not yet fully built technically:** individual components of the arbitration logic [ADR-253]. **A pure proposal, not present in today's system:** the XID component for the identity layer — a targeted search across the entire codebase found not a single genuine match [Canister-Code-Audit] — as well as the biometrically supported Human Identity component, which does not yet exist even as a first test version. These three categories are never mixed: a proposal is never presented as a finished component. What is documented, by contrast, is the economic promise: the core operations — buying, selling, using escrow — remain permanently fee-free under the Colabonate business model, unlike the 15–30% commission charged by conventional online marketplaces [WP7 §4.1].

0.2 In-Depth Summary for the Technical Audience

International payments are undergoing a regulatorily mandated migration to the ISO 20022 messaging standard. This paper argues that ISO 20022 is a necessary but not sufficient standardization: the standard governs exclusively the *message layer* (i.e., how payment data is described and transmitted) and deliberately leaves out three areas that are decisive for economic settlement security in international trade: the *cryptographic verifiability of counterparties*, *dispute-and-arbitration logic*, and *technical settlement* including the escrow function.

The central thesis of this paper is: the Colabonate stack does not replace ISO 20022, but **complements** the standard with exactly these three layers. An ICP-based escrow canister handles non-custodial escrow-and-release logic on native Bitcoin (settlement gap); a Nostr-based identity layer plus a proposed XID connector (Blockchain Commons) supplies verifiable party identifiers (identity gap); a multi-stage, human-led dispute resolution protocol closes the arbitration gap. The technical core and novelty lie in the *integration* of **hash-based payment settlement** (every state change authorized by a BIP-340 signature over a domain-separated hash) and a **non-custodial arbitration mechanism** (off-chain deliberation through a human process, on-chain execution purely via signature verification). This integration resolves the classic dilemma between custody risk and dispute recourse: a cross-border transaction obtains irreversible settlement guarantees *and* binding dispute recourse, without any party ever holding custody. All three additions are designed as *layers on top of* ISO 20022. The standard itself remains untouched and is, operationally, even strengthened.

The paper is honest about implementation status in its claims: where the stack has already been verified end-to-end against `bitcoind -regtest` (a 3-milestone escrow trade split 25/50/25), this is presented as demonstrated proof; where components are still at the stage of an `accepted` design decision or an explicit proposal — notably the XID

connector (not present in the reference implementation) and Human Identity (concept stage, PoC still open) — this is clearly marked as a draft/proposal and not presented as a finished component. The economic claim (permanently fee-free core transactions versus 15–30% commission at Web2 marketplaces) is a documented part of the Colabonate business model.

1. Background and Objective

1.1 Context: The ISO 20022 Migration

International payments are currently transitioning from the classic MT message format (SWIFT) to the richer, XML-based ISO 20022 standard. This migration is not optional; it is regulatorily anchored and, for the most part, already completed: the US Fedwire Funds Service successfully brought the ISO 20022 format live on **July 14, 2025** [FRB], after the cutover originally planned for March 10, 2025 was postponed; the Clearing House Interbank Payments System (CHIPS) had already migrated to ISO 20022 in **April 2024** [TCH]. For SWIFT cross-border traffic, industry sources broadly agree that the coexistence of MT and MX messages ends in **November 2026** with the retirement of category-1/2 MT messages (including MT103, MT202) *[SWIFT data is common industry knowledge; it could not be verified against a reachable primary source in this research — see Section 2.2]*. ISO 20022 thus delivers structured message formats with substantially richer data fields and improved compliance-checking capability than the predecessor standard.

1.2 Objective of This Paper

The goal of this work is to show how *structural* gaps in pure message standardization can be closed by a complementary, decentralized stack. The argument is aimed at economic value for international trade, not primarily at technical interoperability. Technical interoperability is a means, not the end goal; the end goal is reducing settlement risk and cost while increasing trustworthiness between trading parties who do not know each other.

1.3 Target Audience

The intended audience is a committee with a mandate over ISO 20022 *[placeholder for the committee's specific name]*. The presentation assumes familiarity with payment messaging formats, but explains the decentralized components (ICP canister, Nostr, Lightning, XID) to the extent needed to make their *complementary* role to the standard understandable.

1.4 Value Proposition

The economic value for international trade derives from three levers: first, reduced counterparty risk through non-custodial, code-based escrow instead of an institutional trustee; second, cost reduction through a business model whose core transactions, including escrow use, are permanently fee-free (in contrast to 15–30% commission at conventional Web2 marketplaces [WP7 §4.1]); third, traceability and unambiguous identification of trading parties via a cryptographic identity layer.

1.5 Core Positioning: The Guardrail for This Entire Paper

The Colabonate stack is consistently presented as a **complementary layer to ISO 20022**: not as a replacement and not as a supersession. This positioning is not rhetorical but factually required: ISO 20022 is regulatorily anchored and becomes mandatory industry-wide from November 2026; a replacement framing would not be acceptable to a committee and would, in fact, be incorrect, since the standard standardizes a different layer than the one the stack addresses (Section 2.4 systematically derives the three concrete gaps). Consistent language throughout is therefore "complements," "closes a gap," and "complementary layer," rather than "replaces" or "supersedes." A note on honesty: ISO 20022 is not mentioned anywhere in the Colabonate reference implementation itself [Canister-Code-Audit]; the complementarity thesis is thus a *contribution of this paper*, not a claim documented in the source code.

2. ISO 20022: Status Quo and Structural Gaps

2.1 What ISO 20022 Delivers

ISO 20022 is a messaging standard for financial message exchange. Its contribution is substantial and should not be understated here: the standard delivers structured, XML-based message formats with substantially richer data fields than the predecessor MT messages, enabling a more fine-grained, machine-checkable description of payments, parties, accounts, and status information. The improved data quality supports compliance checks (anti-money-laundering, sanctions screening, regulatory reporting) considerably better than the terse, free-text-heavy MT fields. Anyone wanting to trace the full scope of the standard against a reference model can find it in the Java reference implementation *prowide-iso20022* [prowide], which models the ISO 20022 dictionary types across more than 60 message families and roughly 20 business areas. This demonstrates that the standard is broadly scoped and deliberately standardizes far more than individual market participants exploit in practice.

2.2 Timeline and Regulatory Pressure Through November 2026

The migration to ISO 20022 is not a recommendation but a binding, already largely completed process. The following are supported by primary sources:

- **CHIPS** (The Clearing House, USA) migrated to ISO 20022 in **April 2024** [TCH]. The exact date (often cited as April 19, 2024) could be confirmed at month-level against the retrieved TCH primary source in this research, but not at day-level.
- **Fedwire Funds Service** (Federal Reserve, USA) brought the ISO 20022 format live on **July 14, 2025** [FRB], after the cutover originally planned for March 10, 2025 was postponed. Per the source, Fedwire payments are immediate, final, and irrevocable (RTGS) [FRB].
- **SWIFT cross-border traffic (CBPR+)**: According to consistent industry reports, CBPR+ went live on March 20, 2023; the end of MT/MX coexistence, with the retirement of category-1/2 MT messages (including MT103, MT202), is expected for **November 2026**. *Both of these SWIFT dates are common industry knowledge but could not be verified in this research against a reachable primary source (swift.com and iso20022.org were not reachable via automated retrieval), and are therefore flagged as not primary-source-confirmed.* [SWIFT-Reservation]

Regulatory pressure is therefore real: anyone who wants to participate in international payments must speak ISO 20022 by the end of the coexistence period at the latest. This very pressure makes it urgent to ask *what* the standard delivers and *what* it deliberately leaves open.

2.3 What ISO 20022 Does *Not* Deliver

As substantial as the contribution of the message layer is, its limits are equally clear. ISO 20022 describes *how* a payment and its parties are represented, but it does *not* define:

- a **technical settlement escrow**: the standard specifies payment messages, but no escrow slot that holds back consideration until confirmed delivery and releases it automatically. The escrow business remains left to external, institutional trustees or bilateral trust assumptions.
- a **decentralized identity layer**: the standard's messages provide for party and account identifiers (Debtor/Creditor blocks with BIC, IBAN, LEI, organization IDs), but define no cryptographic method to verify the *identity behind* these fields themselves or to trace their history. The identifiers are descriptive, not verifying.
- **automated arbitration**: the standard has status codes (including `RJCT`, `PDNG`, `ACSC` [provide: `TransactionIndividualStatus3Code`]) that reflect the *message state* of a

transaction, but no logic to bindingly decide a substantive dispute between the parties over delivery or performance.

2.4 Deriving the Three Core Gaps

From the limits identified in Section 2.3, three structural gaps emerge that form the point of entry for the Colabonate stack:

- **Gap (a): counterparty identity verification:** the standard describes identifiers but does not verify who stands behind them.
- **Gap (b): native arbitration/dispute logic:** the standard reports status but does not resolve substantive disputes.
- **Gap (c): technical settlement/escrow:** the standard transmits the payment instruction but holds no consideration in trust until performance.

These three gaps structure Sections 4 through 6 of this paper: Section 4 (escrow) closes gap (c), Section 5 (identity layer with Nostr and XID) closes gap (a), and the dispute resolution protocol described in Section 4.4.1 closes gap (b).

2.5 Supplementary Note on Gap (a): The Cryptographic Depth

Within identity gap (a) lies a specific sub-aspect that is highly relevant for international trade: the *lack of cryptographic verifiability and provenance* of party identifiers. The standard can carry an LEI or an organization ID, but it cannot prove that this assignment belongs to a real, key-controlling entity, nor make the history of that assignment cryptographically traceable. It is precisely here, at the cryptographic depth of the identity gap, that the proposed XID connector (Section 5.5) has the greatest leverage. It is important to separate this sub-gap from the *other* identity aspects (uniqueness/sybil resistance, trust metric) that Colabonate addresses conceptually through Human Identity (HID) and the Reputation Scoring Framework (Section 5.1.1). XID complements these but does not overlap with them.

3. The Colabonate Stack: An Overview

3.1 Basic Concept

Colabonate is an open protocol for *Decentralized Collaborative Commerce (DCC)*, governed by the Colabonate Codex and the Colabonate DAO [WP7 §1.1, §2.1]. The platform operates as a decentralized commerce and collaboration platform on Bitcoin L2; its core principles are decentralization, transparency, security (Bitcoin L2, smart contracts, Human Identities), sustainability, and autonomy/self-determination

[WP7 §2.1]. For the economic-value argument to the committee (cf. Sections 1.4 and 7.2), one concrete, citable finding is central: Colabonate's core transactions, *including* escrow use, are, per the business model, **permanently fee-free**: there is no percentage commission and no listing or payout fee [WP7 §4.1]. An arbitration fee (1% for mediation, 2% for arbitration) applies only in an actual dispute. Compared with 15–30% commission at conventional Web2 marketplaces, this is a concrete, documented economic contrast that is taken up again in Section 7.2.

3.1.1 Note on Cross-Version Honesty

The Colabonate Whitepaper, version 7 (August 1, 2026), still describes in §3.3 a three-layer architecture with RSK as the contract layer for escrow/governance (planned for "Phase 4"), and in §5.1 describes Lightning hold-invoice escrow as an already-implemented mechanism. This description is superseded by the architecture decisions ADR-253 and ADR-254, written three days later (August 2–4, 2026): RSK as the contract layer was discarded as a v2 approach, and the custodial Lightning hold-invoice path remains behind a feature flag and, per founder decision, never goes into production.

This time-staggering gives rise to a deliberate methodological source hierarchy for this paper: for the *technical description of the escrow mechanism*, **ADR-253/254 and the referenced code are the sole authoritative source of truth**, because they reflect the currently built and verified state. The Whitepaper v7, by contrast, retains its validity for *context, vision, values, and business model* — everything that makes no claim about the specific escrow mechanism. This separation is applied consistently throughout the paper: every technical claim about escrow, settlement, and arbitration is documented against ADR status and code; every economic or visionary statement is documented against the Whitepaper. The openly noted discrepancy is thus not a charge of inconsistency, but the traceable evidence of active development documented within a matter of days: the Whitepaper simply has not yet been updated to the ADR-253/254 state.

3.1.2 Reference Implementation and Core Decision

The core architectural decision is recorded in ADR-253 (Status: *accepted*, August 2–3, 2026) and reads, pointedly: "*Two paths, zero custody, one new technology*" [ADR-253]:

1. **Direct-payment path** (Lightning or Cashu, non-custodial from day one) for micro-trades without an escrow need.
2. **Escrow path** via an ICP canister on native Bitcoin (for higher-value trades: collaborations, milestones, multi-day deliveries).

Custodial predecessor architectures, platform LND hold invoices (ADR-124/184/193) and the RSK swap sandwich (v2), were explicitly discarded in favor of the ICP approach. The

decisive technical reason: ICP canisters offer native Bitcoin threshold cryptography (t-ECDSA) and therefore require no ckBTC, no mandatory ICP wallet for users, and no platform custody [ADR-253].

3.2 Architecture Building Blocks Overview

The stack consists of three architectural building blocks:

- **(a) ICP escrow canister** (`packages/icp-escrow-canister`): a Rust/Candid canister for settlement and arbitration; the detailed design is documented in ADR-254. This building block is the subject of Section 4.
- **(b) Nostr as the identity and transport layer**: app- and server-side code (`apps/colabonate-app/src/lib/nostr.ts` , `apps/server/services/nostr- .ts`) plus a dedicated schema package (`packages/nostr-schema`). This building block is the real basis of the identity layer (Section 5.1) and of the arbitration transport.
- **(c) Lightning/Cashu as the payment layer**: a payment-provider abstraction (`packages/payments` with LNBits, LND hold invoices, Lightspark, mock) plus a Cashu wallet (`packages/cashu-wallet`) for non-custodial eCash (NIP-60/61). This building block serves the direct-payment path (Section 6).

3.3 Positioning Against Classic Escrow

Unlike classic escrow in international trade, Colabonate has no human or institutional trustee. Control lies with reproducibly built, *blackholed* or DAO-controlled canister code [ADR-253, "Verifiability" section]. Payouts are traceable on-chain rather than booked internally. This property (an escrow function without an escrow agent) is the core of the contribution to settlement gap (c) and is unpacked technically in Section 4.

3.4 Explicit Delimitation: A Complementary Layer Above ISO 20022

The stack does not compete with ISO 20022; it appears as a complementary layer *above* the standard, per the positioning and language convention established in Section 1.5. The following three sections show how this complementary layer closes gaps (a) through (c), derived in Section 2.4, one by one.

4. Escrow Strategy via ICP Canister

4.1 Basic Idea and Demonstrated Proof

The escrow solution is developed in-house and *actively* maps settlement and arbitration via ICP canisters and the Bitcoin blockchain, instead of engaging a central escrow agent.

The source is the package `packages/icp-escrow-canister` (Rust/Candid), whose detailed design is documented in ADR-254 (Status: *accepted*, August 4, 2026) [ADR-254].

The demonstrated technical proof is concrete and reproducible: the canister was verified end-to-end against `bitcoind -regtest`, including a **3-milestone test trade split 25/50/25** with independently checked on-chain balances [Canister-README]. Milestone shares are expressed in basis points (the `milestone_bps` vector must sum to 10,000; the example configuration is `vec{2500, 5000, 2500}` [Canister: `state_machine.rs`]). In the documented test run, the trade was funded with 150,000 sats; after release of all three milestones, cumulative seller balances of 37,500 / 112,500 / 140,042 sats resulted, with escrow change remaining at 109,116 / 30,732 / 0 sats and a total of 9,958 sats in miner fees; the `funded_utxos` reported by the canister matched, at every step, the chain state independently checked via `bitcoin-cli scantxoutset` [Canister-README]. This is a real, replicable proof of settlement functionality, not a concept paper.

4.2 Sequence of an Escrow Transaction

The escrow sequence rests on two consistent principles that precisely qualify it, in a technical sense, as *non-custodial* and *hash-based*. **First**, every state-changing step is authorized by a cryptographic signature *over a deterministic, domain-separated hash*, never by a platform identity (no `msg.caller` auth). Authorization is thus a proof of knowledge of a private key over a computed hash, verifiable on-chain by the canister. **Second**, the actual movement of value (the settlement) takes place via threshold ECDSA: the canister derives a trade-specific Bitcoin key and signs payout transactions *only* after a successful output check. From these two principles it follows that no party, including Colabonate, ever gains control over the escrowed funds. Concretely, the sequence breaks down into the following authorized state transitions:

4.2.1 Trade Creation. The call `create_trade(buyer_pubkey, seller_pubkey, buyer_refund_address, seller_payout_address, expected_amount_sats, milestones, timeout_at)` is triggered by the Colabonate server as an ICP-principal-authenticated call; the trade terms come from a prior Nostr negotiation (Offer/Accept, event kinds 16/17). The canister then derives, via threshold ECDSA, its own trade-specific native P2WPKH Bitcoin address (`derivation_path = [trade_id]`) and enters the `PendingFunding` state.

4.2.2 Funding Check / Deposit. The buyer pays on-chain BTC to the trade address. The call `check_funding(trade_id)` queries the IC's native `bitcoin_get_utxos` interface (no external oracle) and, once a sufficient amount and confirmation depth are reached, sets the state to `Funded`.

4.2.3 Release. `release_milestone(trade_id, milestone_index, buyer_signature)` is exclusively buyer-authorized (the buyer confirms partial delivery or partial performance). The milestone shares in basis points must sum to 10,000; the last milestone sets the state to `Released`.

4.2.4 Arbitration. `open_dispute(trade_id, opener_signature)` is buyer- or seller-authorized, reachable from `Funded`, and locks the regular release/refund calls. `submit_dao_verdict(trade_id, buyer_bps, seller_bps, dao_signature)` checks the signature against a compile-time-fixed DAO public key and triggers the final split payout.

4.2.5 Timeout Path. `claim_timeout_refund(trade_id)` is *permissionless*, callable by anyone (e.g., a keeper script), and refunds the full remaining UTXO to the buyer address, as long as the state is still `Funded` and `now() > timeout_at` holds.

4.2.6 Authorization Without an ICP Wallet Requirement. Buyers and sellers hold only Nostr keys (secp256k1/BIP-340), not ICP principals. State-changing calls that authorize a party (`open_dispute`, `release_milestone`) are verified by the canister itself against a BIP-340 Schnorr signature over a domain-separated message hash (`"colabonate-escrow-v1::[...]"`); there is *no* `msg.caller` auth [ADR-254 §2; Canister: `src/sig.rs`].

4.3 The Novel Integration: Hash-Based Settlement and the Non-Custodial Invariant

Classic escrow faces a structural dilemma: custodial escrow *enables* dispute resolution but creates custody risk (counterparty risk from the escrow agent); pure on-chain escrow *avoids* custody but offers no dispute recourse. The Colabonate approach resolves this dilemma by cleanly separating *custody* from *adjudication* and cryptographically securing both. The integration rests on two pillars.

Pillar 1: Hash-based payment settlement. Every state-changing operation (`release_milestone`, `open_dispute`, acceptance of the DAO verdict) is authorized via a BIP-340 Schnorr signature formed *over a domain-separated message hash* (`"colabonate-escrow-v1:::"`) [ADR-254 §2; Canister: `src/sig.rs`]. The domain separator binds each signature to *one* action on *one* trade, ruling out replay attacks and the reuse of a signature for a different action. Authorization is thus *hash-based*, not *identity-based*: instead of a platform identity (`msg.caller`), the canister requires proof that the caller knows the private key corresponding to a public key. Buyers and sellers authenticate exclusively via their Nostr keys (secp256k1/BIP-340), without an ICP wallet or platform account. The actual settlement then takes place via threshold ECDSA: the canister derives its own Bitcoin key per trade and signs the payout transaction only after the output amounts have been programmatically checked.

Pillar 2: The non-custodial invariant. The formally stated security property is: *no runtime-mutable third recipient*. Every payout signed by the canister may contain outputs solely to three addresses fixed at trade creation: buyer refund, seller payout, and fee (the fee is a compile-time constant, set to 0 in the pilot). This third address cannot be changed at runtime, and there is *no* admin function that overrides this [ADR-254 §5; Canister: `validate_payout_outputs` in `src/state_machine.rs`, checked before

every signing]. Precisely stated, the invariant is "*no runtime-mutable third recipient*," not "*no third address whatsoever*." Combined with Pillar 1, this means the funds are *mathematically bound* at every point in time: held by code that cannot redirect them, authorized only by signatures that only the entitled party can produce.

Why the integration is novel. Because custody (non-custodial canister) and dispute resolution (a four-stage, human process, Section 4.4.1) are decoupled, a cross-border transaction gains *both at once*: irreversible settlement guarantees *and* binding dispute recourse — without any party ever holding custody. Trust is required only for the *quality* of the dispute resolution, never for custody of the funds. Compared with classic escrow, this yields three advantages: automation, drastically reduced counterparty risk, and full on-chain transparency.

Verifiability and decentralized control. The invariant is not merely asserted but verifiable: a reproducible build (`Dockerfile` + `verify-reproducible-build.sh`), an on-chain-retrievable module hash, and a *blackholed* or DAO-multisig-controlled controller guarantee that no party, including Colabonate, can unilaterally alter the code after the fact [ADR-253]. This decentralized control is the philosophical core: authority does not rest with an institution, but with reproducibly built, DAO- or community-controlled code.

4.3.1 Managing Fiat-Crypto Volatilität and Currency Conversion

In international merchandise trade, contracts are predominantly denominated and accounted for in sovereign currencies (EUR, USD). Because the ICP escrow canister natively computes in integer satoshis and multi-week delivery or performance windows are common, exchange-rate volatility between Bitcoin and fiat currencies represents a practical challenge. Colabonate addresses this via a three-tier model:

1. **Fixed-Satoshi Baseline (Crypto-Native Accounting):** For trading partners using Bitcoin as their unit of account, a direct satoshi amount is fixed at trade creation. No currency risk arises; 100% of the agreed sats are paid out according to the milestone schedule.
2. **Dual-Denominated Escrow with Buffer (Target Fiat Hedging):** For contracts denominated in EUR/USD, the buyer deposits a satoshi equivalent representing a 110–115% buffer relative to the exchange rate at trade creation. Upon each milestone release (`release_milestone`), the canister queries a decentralized exchange-rate oracle to compute the exact satoshi amount matching the due fiat tranche and pays it out to the seller. After the final milestone is fulfilled, the remaining unused buffer is automatically refunded to `buyer_refund_address` in the same transaction (`validate_payout_outputs`).
3. **Multi-Token and Stablecoin Roadmap (ckUSDC / ckEUR):** Through native multi-token support on the Internet Computer, the canister architecture can be extended to

chain-key tokens (ckUSDC, ckEUR). This achieves full fiat parity without crypto volatility, while preserving non-custodality and BIP-340 signature authorization.

4.4 Interface to ISO 20022: An Honest Mapping

Where in the payment flow does the canister logic engage? Stated precisely and honestly: the canister does not replace *any* ISO 20022 message; it replaces the *custody-and-release logic* that follows a payment already described in an ISO-20022-compliant way. There is *no* 1:1 mapping between canister states and ISO 20022 status codes, for a substantive reason: canister states are technical escrow states of a trust UTXO, whereas ISO 20022 status codes reflect message states of a payment transaction. These are different axes. The appropriate correspondence is therefore a *temporal sequence with economic correspondence*, not a tabular equivalence:

Canister State	Economic Correspondence	ISO 20022 Correspondence (Conceptual)
PendingFunding	Trade opened, deposit pending	triggered by a pacs.008 credit transfer (Customer Credit Transfer); canister awaits the funding UTXO
Funded	Deposit confirmed, escrow active	economically close to ACSP / ACSC (accepted for settlement / settlement completed on the debtor side)
Released	Consideration delivered, payout to seller	economically ACSC (settlement completed) on the creditor side
Disputed	Dispute, regular release locked	<i>no direct normative analogue</i> : the standard has no arbitration state (gap b); this is exactly the addition the stack provides
Refunded	Timeout- or verdict-driven repayment	economically related to a chargeback/ RJCT direction

The normative status codes come from the enum `TransactionIndividualStatus3Code` in the pacs.002 family (values including `ACTC`, `RJCT`, `PDNG`, `ACCP`, `ACSP`, `ACSC`, `ACWC`) [provide: `model-common-types/.../TransactionIndividualStatus3Code.java`]. Two observations sharpen this picture. First, the ISO 20022 standard is considerably broader than its practical use. A widely used JS implementation, *iso20022.js*, covers only about seven message families across two business areas (roughly 12% of the standard's breadth as seen in the provide model) and does not implement the FI-to-FI status reports pacs.002/pacs.008 *at all*: only the customer-to-bank side pain.002, with six codes [iso20022.js: `src/pain/002/types.ts`]. Second — and this is the decisive, and honestly stated, finding — the Colabonate reference implementation does not reference ISO 20022 anywhere: a search across the entire canister package yields zero hits, and the Candid interface carries exclusively crypto-native fields (public keys, addresses, sats, basis points, timeout), no remittance, debtor/creditor, or currency fields [Canister:

`icp_escrow_canister.did`]. The table above is therefore a *conceptual correspondence established by this paper*, not an ISO 20022 binding present in the code. What is still missing for a complete, ISO-20022-related PoC is a message adapter that translates `pacs.008` fields into `create_trade` parameters and, conversely, reports canister states back as `pacs.002` status reports (Sections 8.3 and 8.4).

4.4.1 The Arbitration Process: Non-Custodial Deliberation and Cryptographic Execution

The key to the non-custodial arbitration mechanism lies in a strict separation of *deliberation* and *execution*. At `submit_dao_verdict`, the canister checks exclusively the *signature* of a verdict. It performs **no** substantive evaluation. How the decision is reached is worked out entirely *off-chain*, in a **four-stage dispute resolution protocol** [WP7 §3.5.5]:

1. structured, on-chain-documented communication between the parties;
2. peer mediation by 2–3 randomly selected users with, *by design*, verified HID and a minimum reputation threshold;
3. subject-matter arbitrators with category expertise, if stage 2 fails;
4. a DAO appeal as the final instance, via vote with a minimum quorum.

The *outcome* of this off-chain deliberation (a percentage split of `buyer_bps` / `seller_bps`) is enforced on the canister *solely* through signature verification against a compile-time-fixed DAO public key. Execution is thus deterministic and cryptographic: the canister makes no subjective decision — it validates only that an authorized key signed the verdict, and then triggers the payout that is already fixed by Pillar 2 (Section 4.3) regardless. This is precisely what makes the arbitration mechanism non-custodial: dispute resolution *requires no custodial power*. Throughout the entire process, the funds remain mathematically bound in the non-custodial canister, and the verdict merely moves them within the predetermined addresses, without opening up any new recipient possibility.

Explicitly *without* automated AI-bot mediation: the Whitepaper's rationale is traceability and fairness [WP7 §3.5.5]. What matters for the committee: the standard's gap — "missing arbitration logic" — is thus not closed by pure code automation, but by a multi-stage, *human-supported* process, whose result flows into the canister in a cryptographically enforceable way. The Reputation Scoring Framework directly links the outcome of the dispute process to the reputation of the parties involved (Section 5.1.1), a compliance-relevant feedback effect.

Honest status note on the HID linkage: HID-verified status for mediators (stage 2) is a *design goal*, not a current state of fact; HID itself has not yet been validated as a proof of concept (status and preliminary work in detail: Section 5.1.1). Until HID validation,

mediator selection is secured via the reputation and review system; the HID linkage remains a development step still to be delivered.

4.4.2 Legal Classification and Arbitrability Under UNCITRAL

Dispute resolution in cross-border B2B commerce is subject to formal legal requirements. To ensure that outcomes of the four-stage arbitration process are not only executable technically but legally resilient against challenges in national courts, the stack integrates into established legal frameworks:

- **Electronic Arbitration Agreement (Art. 7 UNCITRAL Model Law):** When counterparties mutually sign trade terms via Nostr (event kinds 16/17), they conclude a legally binding arbitration agreement in electronic form pursuant to Article 7(4) of the UNCITRAL Model Law on International Commercial Arbitration. The hash of the trade terms (`trade_terms_hash`) stored on-chain proves mutual submission to the dispute protocol.
- **Qualification as Expert Determination (*Schiedsgutachten*):** The determination by mediators or the DAO regarding the percentage payout split (`buyer_bps / seller_bps`) legally qualifies as a predetermined contractual adjustment mechanism or expert determination.
- **Deterministic Execution Without Judicial Enforcement Procedures:** Because funds are already escrowed in the canister and released purely through cryptographic signature checks, the necessity of lengthy exequatur proceedings under the 1958 New York Convention is eliminated. The verdict takes immediate economic effect.

4.5 Honest Implementation Status

Honesty before the committee requires that no design decision be presented as a finished implementation. ADR-253 itself runs an "Implementation Reality Check" with open items, stated here without embellishment [ADR-253]:

- The **DAO governance key is not yet configured**: `submit_dao_verdict` deliberately returns an error instead of a stub success response (`sig::DAO_PUBKEY_HEX = None`). The dispute path therefore exists in code but is *currently not fully usable* until the DAO key is set.
- **Canister state still resides in heap rather than stable memory**, an upgrade risk before mainnet.
- The **blackhole-vs-DAO-controller decision remains open** (follow-up FU-253-H).
- The existing **custodial hold-invoice path (Lightning)** remains behind a feature flag in the code but, per founder decision, will *never* be publicly activated.

These points are not a weakness of the approach but evidence of an honestly maintained implementation status: they are carried forward in Section 7.1 as the respective implementation status (`partial`) per component.

4.6 Legal and Regulatory Safeguards

The architecture is designed from the outset so that decentralized philosophy and legal protection are not mutually exclusive but mutually reinforcing. The following safeguards arise *structurally* from the design; they are not after-the-fact promises but properties of the built code.

- **Non-custodality as protection against money-transmitter exposure.** Because the Colabonate entity never holds control over the escrowed funds and cannot alter the third payout recipient at runtime (Pillar 2, Section 4.3), the architecture is designed so that no custodial money-transmitter role for the platform arises in the escrow leg. The technical settlement agent is the non-custodial canister, not Colabonate.
- **Auditable code as a basis for accountability.** A reproducible build, an on-chain-retrievable module hash, and a *blackholed* or DAO-multisig-controlled controller [ADR-253] make the exact code governing the funds verifiable by every party and protect against subsequent, unilateral manipulation. Responsibility is thereby distributed and transparent, not centralized.
- **Permissionless timeout as protection against fund lock-up.** The timeout path (`claim_timeout_refund` , Section 4.2.5) can be executed by any caller. Even in the event of a complete platform outage, funds cannot be permanently locked — an operational and legal safety-valve function against indefinite custody.
- **Due-process-consistent, on-chain-documented arbitration.** The four-stage dispute resolution protocol (Section 4.4.1) is conducted with on-chain documentation; mediator selection is, *by design*, HID-verifiable and reputation-bound, and the verdict feeds deterministically into reputation. This supports a traceability-and-fairness argument and makes the course of the dispute auditable.
- **Privacy by design.** The zero-knowledge design of HID (no raw-data storage, concept stage; see Section 5.1.1) and XID elision (selective disclosure without breaking the signature, Section 5.5.2) are designed for data minimization and are therefore relevant to cross-border KYC/privacy requirements.
- **No identity coercion.** Users transact exclusively with their own Nostr keys; there is no mandatory ICP wallet and no custodial onboarding requirement. Self-determination of the parties over their identity and their money is preserved.

Limiting clarification: the points above are *architectural properties*, not legal assessments. The regulatory classification — in particular, whether and how a given

jurisdiction qualifies the canister, the DAO, or the Colabonate entity — is jurisdiction-dependent and must be confirmed by qualified legal counsel before any production rollout. This paper presents the *technical* protective posture, not its legal recognition.

5. Identity Layer: Nostr, XID, and the Trust Model

5.1 Nostr as the Base Layer (Implemented)

The real basis of the identity layer is Nostr, a key-based, censorship-resistant protocol for communication and identity. Unlike the proposed components further below in this section, the Nostr integration in the Colabonate reference implementation is **real and broadly built out**, not merely conceptual: on the app side there are modules for event building/publishing, kind:0 profiles, NIP-59 gift wrap (kind:1059), NIP-17 private direct messages (kind:14), identity, handshake, NIP-44 encryption, NIP-09, and NIP-89 (`apps/colabonate-app/src/lib/nostr*.ts`, `nostr-identity.ts`, `nostr-handshake.ts`, `nip44.ts`, `nip09.ts`, `nip89.ts`); on the server side, a relay service, Nostr auth, DAO-Nostr, and corresponding routes (`apps/server/services/nostr-relay.ts`, `nostr-auth.ts`, `dao-nostr.ts`; routes `auth-nostr.ts`, `identity.ts`, `dao.ts`).

Standard NIPs in use are 01 (base protocol), 42 (relay auth), 44 (encryption v2), 59 (gift wrap), 46/07 (remote signer/bunker), 98 (HTTP auth, 60-second replay window), 60/61 (Cashu wallet), and 17 (private DMs). Beyond these, Colabonate defines its own addressable event kinds 30017–30027 for protocol-specific objects (ADR-009, status: *implemented*): 30017 Offer, 30018 Ticket Created, 30019 Ticket Status Update, 30020 Dispute Opened, 30021 Verification Credential ("Soulbound"), 30022 Governance Vote, 30023 HID/Humanode Attestation, 30024 Reputation Rating, 30025 Token Stake, 30026 Proximity Proof, 30027 Company Profile [ADR-009], documented as an internal convention pending official NIP registration.

5.1.1 Supplementary Identity and Trust Components (Concept/Draft Stage)

Two components supplement the Nostr base and matter for the argument to the committee, because they close *different* dimensions of identity gap (a) than XID and must not be conflated with it. Both are currently *concepts*, not built components: their inclusion here is a draft-stage disclosure, not evidence of implementation.

- **Human Identity (HID)**, with optional biometric linkage via the Humanode Biomapper [WP7 §3.4]: a mechanism *designed* as a zero-knowledge uniqueness proof without raw-data storage, with the design goal "*one real person = one identity*."
Honest status: HID has *not yet been developed or tested* as a proof of concept and still needs to be validated before it can be relied on to reliably establish the

authenticity of a human identity. The relevant *preliminary work*, however, is in place: the specification in the Whitepaper [WP7 §3.4], the addressable Nostr event kind 30023 ("HID/Humanode Attestation," ADR-009), and the integration interface to the Humanode Biomapper. If successfully validated, HID addresses the *sybil/uniqueness problem*.

- **Reputation Scoring Framework (RSF)** [WP7 §3.5]: deliberately *designed* as deterministic and rule-based rather than an AI black box, with six auditable dimensions (reliability, quality, communication, compliance, fairness, community contribution) and DAO-controlled weights via 1P1V voting. RSF is designed as a concept to deliver a *traceable trust metric* between trading partners; today's real trust anchor in the stack is the Nostr-based reputation/review system (kind 30024).

Both components complement each other conceptually but do *not* overlap with what XID would provide (cryptographic provenance and history of individual identifier attestations, Section 5.5). In the comparison table in Section 7.1, they appear as separate dimensions alongside XID.

5.2 XID (Blockchain Commons) as a Proposed Extension

XID (*eXtensible IDentifier*, Blockchain Commons) is a self-sovereign, provenance-based identifier with selective disclosure via Gordian Envelopes. The status here must be marked with complete clarity: **XID is currently neither implemented nor referenced in the Colabonate reference implementation.** A targeted search for "XID," "Gordian Envelope," "Provenance Mark," "bc-envelope," and "did:key" across the entire repository found no genuine matches (only false positives such as `taxId / txid`) [Canister-Code-Audit]. The app's real identity layer today is Nostr keys plus reputation/review events (kind 30024), *not* a W3C DID or XID format. This section is thus to be read as a **complementary proposal for the Colabonate stack**, not as a description of a built component. The argument to the committee nonetheless remains valid (XID closes a gap that neither ISO 20022 nor today's Colabonate stack closes); only the wording must cleanly preserve the distinction between *proposed* and *implemented*. Primary sources are the Blockchain Commons specifications BCR-2024-010 (XID) and BCR-2026-003 (XID Edges) [BCR-2024-010; BCR-2026-003].

5.3 Complementarity of Nostr Keys and XID

The combination of both is stronger than either component alone, because it builds on the *same* cryptographic substance: what exists in today's stack is only the Nostr-key side (secp256k1/BIP-340; see Section 5.1 and 4.2.6 on canister-side signature verification). The XID side would be an *additive extension on the same keys*, not a replacement. Concretely, an XID is deterministically formed as the SHA-256 hash of the CBOR encoding of a `PublicSigningKey` (inception key), and is thus bound to the same

key pair, while remaining stable over its lifecycle, since verification keys can be rotated without changing the XID [BCR-2024-010].

5.4 Use Case in International Trade

The use case is verifying trading partners without a central issuing authority. Today's actual state in the reference implementation is a pragmatic intermediate step: relay ownership verification via a `kind:0/kind:30021/kind:30405` credential (an "L1 upgrade") plus NIP-98 auth (`apps/server/routes/identity.ts`). This step could be deepened by XID (provenance proof, selective disclosure), *without* requiring the current system to be replaced.

5.5 XID as a Standalone Complementary Component to ISO 20022

ISO 20022's messages provide for party and account identifiers, but, as laid out in Section 2.3, they define no cryptographic method to verify the identity behind these fields or to make its history traceable. It is precisely this gap that the proposed XID connector (cf. status in Section 5.2) *would* close, via three mechanisms:

- **5.5.1 Provenance Marks:** a traceable, hash-chain-based history of an identity over time, including proof of which version is currently valid. Relevant for compliance checks in payments.
- **5.5.2 Gordian Envelope / selective disclosure (elision):** trading partners can selectively disclose or conceal individual attestations, *without* breaking the signature of the overall document. Relevant for privacy in cross-border KYC requirements.
- **5.5.3 Key-based self-sovereignty without a central issuing authority:** structurally consistent with the decentralized canister and Nostr identities in the stack, unlike classic, centrally issued identifiers.

5.6 Concrete Proposal for ISO 20022 Message Fields

Building on the provide field analysis [provide] and the XID specification [BCR-2024-010], a concrete, *minimally invasive* mapping proposal can be formulated. ISO 20022 already has a normatively provided extension mechanism for proprietary identification schemes: the generic slot `0thr` (e.g., `GenericOrganisationIdentification1`) within the debtor/creditor party blocks, with the free-text sub-elements `Id`, `SchmeNm` (either an official code `Cd` or a *proprietary* string `Prtry`), and `Issr`. An XID reference can be embedded there *additively*, without altering existing field semantics:

```

<Dbtr>
  <Nm>Acme Cooperative</Nm>
  <Id>
    <OrgId>
      <Othr>
        <Id>529900T8BM49AURSD055</Id>
        <SchmeNm><Prtry>LEI</Prtry></SchmeNm>
        <Issr>GLEIF</Issr>
      </Othr>
      <Othr>
        <Id>71274df1</Id>
        <SchmeNm><Prtry>XID</Prtry></SchmeNm>
        <Issr>Blockchain Commons</Issr>
      </Othr>
    </OrgId>
  </Id>
</Dbtr>

```

This is *not* a change to the standard, but the intended use of the normative `0thr / Prtry` extension point: `0thr` is an array, the XID entry is placed *alongside* existing identifiers (LEI, BIC, internal IDs), and all fields remain optional: a recipient that does not recognize the "XID" scheme simply ignores the entry (forward-compatible). Analogous paths exist for persons (`PrvtId/0thr`), accounts (`AccountIdentification4Choice.0thr`), and financial institutions (`FinInstnId/0thr`) [prowide].

Honest limitation (length/encoding constraint). A complete, machine-verifiable XID (32 bytes) does not fit, in any standardized encoding, into a `Max35Text` field: as hex it is 64 characters, as `ur:xid/...` roughly 85 characters, and as a full XID document (Gordian Envelope) several hundred characters [BCR-2024-010]. The `0thr.Id` field can therefore hold *only a verifiable pointer*, not the full proof. The recommended approach for a prototype is to carry the 4-byte recognition prefix of the XID (e.g., `71274df1` , 8 characters) in `0thr.Id` as a recognition hint, and to perform full verification (across all 32 bytes) as well as resolution via a `dereferenceVia` URL *out of band* [BCR-2024-010]. The specification explicitly clarifies that machine comparison always occurs across all 32 bytes, and that the 4-byte prefix alone is collision-prone and not proof. The ISO 20022 message thus carries the *hint*; the cryptographic *proof* remains an additive, decentrally maintained layer — consistent with the complementarity thesis of this entire paper.

5.7 End-to-End Resolution Process for XID and Nostr Identities

For enterprise ERP systems and banking gateways, this results in a straightforward, four-step verification lifecycle:

1. **Extraction of Identifier Pointer:** The receiving enterprise system receives the ISO 20022 XML message and extracts the 4-byte prefix (e.g., `71274df1`) from

`Dbtr/Id/OrgId/Othr[SchmeNm/Prtry='XID']/Id`.

2. **Out-of-Band Resolution:** Via the Nostr network (querying kind:30027 company profile events) or a decentralized resolver, the full 32-byte XID Inception Document (Gordian Envelope) is fetched.
3. **Cryptographic Signature & Provenance Verification:** The system verifies the BIP-340 Schnorr signature of the inception key and validates the hash chain (Provenance Marks) up to the current state.
4. **Selective Disclosure (Elision):** Using Gordian Envelope technology, the system validates the revealed identity claims (e.g., LEI `529900T8BM49AURSD055` against the GLEIF registry), while private corporate metadata remains cryptographically redacted without invalidating the root envelope signature.

6. Lightning as the Payment Layer

6.1 Role in the Stack (Implemented, Partially Legacy)

Lightning serves in the stack as a fast, low-cost Bitcoin L2 payment rail, *complementary* to the escrow layer. The reference implementation abstracts payment providers behind the `PaymentProvider` interface (`packages/payments/src/PaymentProvider.ts`), with the operations `createInvoice`, `createHoldInvoice` / `settleHoldInvoice` / `cancelHoldInvoice`, `payInvoice`, `createSubWallet`, and concrete implementations `LNBitsProvider`, `LndHoldClient` (native LND hold invoices, ADR-193), and `LightsparkProvider`.

The status of the hold-invoice path must be clarified: it is the **legacy/custodial** escrow rail and remains behind a feature flag; per founder decision, it will *never* be publicly activated once the ICP canister is in production [ADR-253]. Direct Lightning payments *without* escrow (the buyer pays the seller's invoice directly, via an LNURL address/NWC) remain, by contrast, the "Stage 1" launch path for micro-trades that need no escrow. As a second non-custodial payment rail for direct payments, Cashu/eCash (NIP-60/61, `packages/cashu-wallet`) is added.

6.2 Interaction with the ICP Canister: Two Rails, No Remote Control

A Lightning-*based* canister escrow was explicitly evaluated and rejected in ADR-253 [ADR-253]. The technical reason is structural: Lightning funds exist exclusively in the channels of a continuously online node; a canister, structurally, *cannot* be a node (no persistent P2P connections, roughly 1-2 seconds of consensus latency). Any construction of the kind "a canister remotely controls an LND" would leave actual control with the node operator and would therefore be custodial. The consequence is a clean separation of rails: Lightning and Cashu serve the *direct-payment path*; the ICP canister

with native Bitcoin threshold cryptography serves the *escrow path*. Both rails coexist; Lightning does *not* technically trigger escrow release.

7. Comparative Overview for the Committee

7.1 Comparison Table (With Implementation Status per Row)

The following table compares "ISO 20022 alone" with "ISO 20022 + Colabonate stack." Decisive for credibility with the committee: the right-hand column carries the *implementation status* per row (`implemented` / `partial` / `proposed`), so the table never promises more than the code delivers.

Category	ISO 20022 Alone	ISO 20022 + Colabonate Stack	Status
Settlement security / escrow	no native escrow; escrow is external/institutional	non-custodial escrow invariant (no runtime-mutable third recipient) + hash-based authorization (BIP-340 over a domain separator), traceable on-chain	partial (verified against bitcoind - regtest [ADR-254])
Sybil resistance / identity uniqueness	identifiers are descriptive only	Human Identity (HID), design goal: 1 person = 1 identity, ZK uniqueness	proposed (concept, PoC open; preliminary work in place [WP7 §3.4])
Trust metric	none	Reputation Scoring Framework (RSF), 6 auditable dimensions, deterministic	proposed [WP7 §3.5]
Identity verification & provenance	no cryptographic verification of fields	XID (Provenance Marks, selective disclosure) as an addition	proposed (not present in code [BCR-2024-010])
Arbitration / dispute logic	status codes, no substantive decision	four-stage dispute resolution protocol, non-custodial (off-chain deliberation, execution via DAO verdict signature)	partial (DAO key not yet configured [ADR-253; WP7 §3.5.5])
Legal Arbitrability	Formal court or arbitral litigation (T+months)	UNCITRAL-compliant electronic arbitration agreement + automatic execution as expert determination	partial (Conceptually resolved, pilot open)
Currency & Volatility Management	Classic bank FX hedging instruments	Dual-Denominated Escrow with buffer + multi-token roadmap (ckUSDC/ckEUR)	proposed (Architectural model)
Speed (settlement)	classic industry norm T+1 to T+3 <i>[not primary-source-confirmed]</i>	on-chain confirmation + immediate release on milestone	partial
Cost	classic: World Bank remittance average 6.36% <i>[low-value P2P corridor]</i> ; wholesale correspondent-banking figure not documentable	core transactions 0% commission; arbitration only 1-2% in a dispute; on-chain fee overhead ~200-1,000 sats/trade (hypothesis)	implemented (business model) / partial (economics gate open) [WP7 §4.1; ADR-253]
Transparency / privacy	transparent messages, no elision mechanism	XID Gordian Envelope elision (selective disclosure)	proposed [BCR-2024-010]

Category	ISO 20022 Alone	ISO 20022 + Colabonate Stack	Status
		without breaking the signature)	
Cross-border suitability	present (message level)	natively cross-border (Bitcoin L2, no correspondent-banking network needed)	partial

7.2 Economic Value Proposition

The core economic advantage is the reduction of settlement risk and cost in international trade. As documented in Section 3.1, Colabonate's core transactions, including escrow use, are permanently fee-free, versus 15–30% commission at conventional Web2 marketplaces [WP7 §4.1]. As a comparison to classic correspondent banking: the World Bank remittance average is 6.36% (Q3 2025) [WB-RPW], with the explicit caveat that this is a *low-value P2P remittance* benchmark (typical test amounts of USD 200–500) and *not* a metric for high-value wholesale correspondent-banking payments, for which no public primary figure exists.

Additionally, from ADR-253: the on-chain fee overhead of ICP-canister escrow (roughly 200–1,000 sats per trade under normal fee conditions, two or more transactions) is the remaining figure still to be measured, and is the subject of the economics gate (FU-253-H) [ADR-253]. For the minimum escrow size (working hypothesis of roughly 100–250k sats), this overhead is genuinely relevant; once measurements are available, they should replace the working hypothesis in the final table. In the documented 3-milestone test, total miner fees were 9,958 sats on 150,000 sats of funding.

7.3 Framing as a Complement, Not a Replacement

For committee acceptance, this framing is decisive: ISO 20022 remains untouched as a messaging standard and is, operationally, even *strengthened*, because identity, arbitration, and settlement are for the first time structurally secured. No change to ISO 20022 itself is required; the XID connector explicitly uses the *already normatively provided* `0thr / Prtry` extension point (Section 5.6), and the escrow operates *above* the message layer.

8. Proof-of-Concept Sketch and ISO 20022 Adapter Blueprint

8.1 Minimal Test Case and Demonstrated Proof

The minimal test case is a cross-border trade transaction with escrow via the canister, identity via Nostr (and, prospectively, XID), payment via Lightning, and an ISO-20022-compliant message layer.

Two things must be honestly separated: **(1) what has already been delivered:** the 3-milestone escrow trade split 25/50/25 documented in Section 4.1, reproducibly verified against an isolated `bitcoind -regtest` node [Canister-README]. This test run is the core technical proof that the non-custodial escrow-and-release logic works. **(2) what is still to be developed for an ISO-20022-related PoC:** the test is a *purely* canister/Bitcoin test *without* any ISO 20022 reference. A complete, ISO-20022-related PoC requires the message adapter specified below.

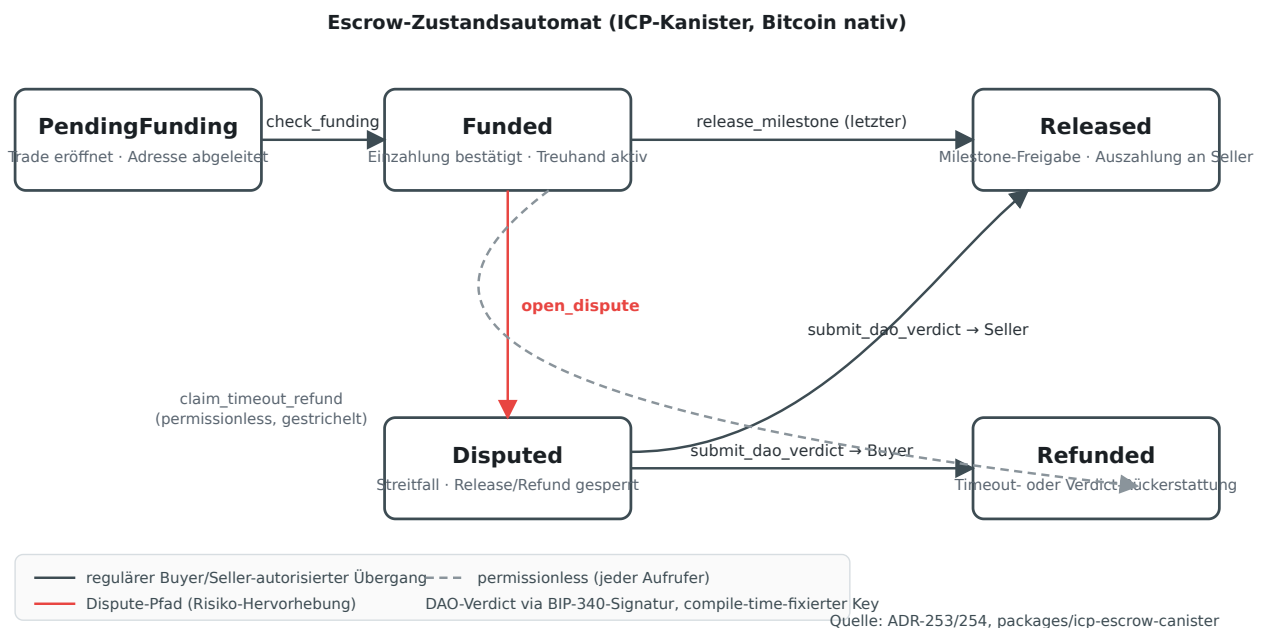


Figure 1: State machine of the ICP escrow canister. Highlighted in red is the dispute path as a risk edge; dashed is the permissionless timeout refund. Source: ADR-253/254, `packages/icp-escrow-canister`.

8.2 Success Criteria (Pilot Gates from ADR-253)

As success criteria for the PoC, this paper adopts the pilot gates defined in ADR-253 [ADR-253]:

- **Economics gate:** measurement of the on-chain fee overhead (two or more transactions per trade) and establishment of a minimum escrow size.

- **Security gate:** external audit (t-ECDSA key handling, UTXO edge cases, reorg depth).
- **Governance gate:** reproducible build documentation and controller status (blackhole or DAO multisig).
- **Pilot gate:** testnet/regtest end-to-end, *including* dispute and timeout refund, followed by a mainnet pilot (≥ 20 trades, ≥ 1 dispute, ≥ 1 timeout).

8.3 Concrete pacs.008 to Candid Adapter Blueprint

To cleanly bridge the gap between traditional banking messages and the ICP canister, a lightweight adapter deterministically translates incoming `pacs.008.001.10` credit transfer instructions into the Candid interface `create_trade` (`packages/icp-escrow-canister/src/types.rs`):

ISO 20022 <code>pacs.008</code> XML Path	ICP Escrow Canister Candid Field	Type & Conversion Rule
<code>CdtTrfTxInf/PmtId/EndToEndId</code>	<code>trade_id</code> (context/mapping)	Unique transaction ID
<code>CdtTrfTxInf/Dbtr/Id/OrgId/0thr[Prtry='XID']/Id</code>	<code>buyer_pubkey</code>	32-byte x-only Nostr/XID Inception Key (Hex)
<code>CdtTrfTxInf/Cdtr/Id/OrgId/0thr[Prtry='XID']/Id</code>	<code>seller_pubkey</code>	32-byte x-only Nostr/XID Inception Key (Hex)
<code>CdtTrfTxInf/DbtrAcct/Id/0thr/Id</code>	<code>buyer_refund_address</code>	Native SegWit Bitcoin address (<code>bc1q...</code>)
<code>CdtTrfTxInf/CdtrAcct/Id/0thr/Id</code>	<code>seller_payout_address</code>	Native SegWit Bitcoin address (<code>bc1q...</code>)
<code>CdtTrfTxInf/IntrBkSttlmAmt</code>	<code>expected_amount_sats</code>	Conversion of transfer amount to satoshis (<code>u64</code>)
<code>CdtTrfTxInf/RmtInf/Ustrd</code>	<code>milestone_bps</code>	Parsing milestone schedule (e.g. <code>MS:2500,5000,2500</code>)
<code>CdtTrfTxInf/SttlmTmReq/TillTm</code>	<code>timeout_at</code>	Expiry timestamp in nanoseconds (<code>u64</code>)

Example of an incoming pacs.008 message fragment:

```

<CdtTrfTxInf>
  <PmtId>
    <EndToEndId>TRADE-2026-08-001</EndToEndId>
  </PmtId>
  <IntrBkSttlmAmt Ccy="EUR">1500.00</IntrBkSttlmAmt>
  <Dbtr>
    <Nm>Buyer Corp</Nm>
    <Id><OrgId><Othr><Id>71274df1</Id><SchmeNm><Prtry>XID</Prtry></SchmeNm></Othr></O
  </Dbtr>
  <DbtrAcct>
    <Id><Othr><Id>bclqbuyerrefundaddressxxxxxxxx</Id></Othr></Id>
  </DbtrAcct>
  <Cdtr>
    <Nm>Seller Global Ltd</Nm>
    <Id><OrgId><Othr><Id>a839f201</Id><SchmeNm><Prtry>XID</Prtry></SchmeNm></Othr></O
  </Cdtr>
  <CdtrAcct>
    <Id><Othr><Id>bclqsellerpayoutaddressxxxxxxxx</Id></Othr></Id>
  </CdtrAcct>
  <RmtInf>
    <Ustrd>/MS/2500,5000,2500/EXP/1787123456</Ustrd>
  </RmtInf>
</CdtTrfTxInf>

```

8.4 Status Feedback via pacs.002

As the canister state progresses through on-chain events, the adapter generates a standardized `pacs.002.001.12` Payment Status Report back to banking systems:

```

<FIToFIPmtStsRpt>
  <GrpHdr>
    <MsgId>STAT-2026-08-001-01</MsgId>
    <CreDtTm>2026-08-06T14:30:00Z</CreDtTm>
  </GrpHdr>
  <TxInfAndSts>
    <OrgnlEndToEndId>TRADE-2026-08-001</OrgnlEndToEndId>
    <TxSts>ACSP</TxSts>
    <StsRsnInf>
      <Rsn><Prtry>ESCROW_FUNDED_ONCHAIN</Prtry></Rsn>
      <AddtlInf>BTC UTXO confirmed at block height 892140</AddtlInf>
    </StsRsnInf>
  </TxInfAndSts>
</FIToFIPmtStsRpt>

```

8.5 Open Technical Questions and Development Roadmap

The code audit and the research conducted for this paper surface the following prioritized engineering tasks:

1. **Standalone Adapter Module:** Implementation of the ISO 20022 XML-to-Candid adapter specified in Sections 8.3 and 8.4 as a standalone microservice repository.
2. **Migration to Stable Memory (FU-254-G):** Transitioning canister state from heap memory to `ic-stable-structures` to guarantee upgrade safety on mainnet.
3. **Provisioning the DAO Governance Key (FU-253-H):** Configuring `sig::DAO_PUBKEY_HEX` in the canister for production readiness of the arbitration path.
4. **End-to-End Test Harness:** An automated test harness that generates a `pacs.008` instruction, funds the canister on Bitcoin regtest, releases milestones via BIP-340 signatures, and verifies the corresponding `pacs.002` reports.

9. Conclusion and Recommendation to the Committee

9.1 Summary of the Central Thesis

The Colabonate stack does not replace ISO 20022; it closes the three structural gaps that the standard deliberately leaves open: *identity verification* of the counterparty, *arbitration* of disputes, and *technical settlement* including escrow. This addition takes the form of a layer *above* the messaging standard, without altering it — in the case of XID, even using the extension point (`0thr / Prtry`) already provided for normatively. ISO 20022 remains untouched and is operationally strengthened.

9.2 Concrete Recommendation for Action

The recommendation to the committee is: evaluate the Colabonate stack as an optional complementary layer, or reference architecture, *above* ISO 20022. No change to ISO 20022 itself is required. Where the stack has already delivered (the non-custodial escrow invariant, verified against regtest; the fee-free business model), the proof is concrete; where it remains a proposal (XID provenance, RSF, the full set of pilot gates), this is openly disclosed and presented as a development roadmap. The next concrete step is the ISO 20022 message adapter (Sections 8.3 and 8.4), which seamlessly connects the already-functioning escrow channel to the standard.

References

Sources are referenced as short tags in the running text and resolved here.

[WP7] Colabonate Whitepaper v7, `docs/colabonate_whitepaper_de_v7.md` (Colabonate app repository, August 1, 2026). Note: Whitepaper v7 predates ADR-253/254 and describes, in §3.3/§5.1, the *discarded* predecessor architecture (RSK, Lightning hold-invoice escrow); for the technical escrow mechanism, ADR-253/254 plus code are treated as the source of truth.

[ADR-253] ADR-253: Non-Custodial P2P Escrow Strategy, `docs/decisions/253-non-custodial-p2p-escrow-strategy.md` (Status: accepted, August 2-3, 2026). Source for: the two-path/zero-custody decision, the implementation reality check, pilot gates, FU-253-H, and the rejection of RSK/Lightning hold-invoice escrow.

[ADR-254] ADR-254: ICP Escrow Canister Detail Design, `docs/decisions/254-icp-escrow-canister-detail-design.md` (Status: accepted, August 4, 2026). Source for: BIP-340 authorization (§2), the output invariant (§5), test strategy (§12).

[ADR-009] ADR-009: Nostr Event Kind Range, `docs/decisions/009-nostr-event-kind-range.md` (Status: implemented). Source for event kinds 30017-30027.

[Canister-README] `packages/icp-escrow-canister/README.md` : source for the 3-milestone test trade 25/50/25, 150,000 sats funding, 9,958 sats fees, `scantxoutset` verification.

[Canister-Code-Audit] Code audit dated August 5, 2026, over `packages/icp-escrow-canister` : no matches for ISO 20022 / XID / Gordian / Provenance Mark / did:key (only false positives `taxId` / `txid`); the Candid interface carries exclusively crypto-native fields.

[provide] `provide-iso20022` (Java reference implementation of the ISO 20022 dictionary types), `Repos/provide-iso20022` . Source for the status-code enum `TransactionIndividualStatus3Code` (`model-common-types/.../TransactionIndividualStatus3Code.java` , 7 values) and the party/account block field structure including the generic `0thr / SchmeNm.Prtry slot`. Detailed analysis: `Research/2026-08-05-provide-statuscode-analyse.md` .

[iso20022.js] `iso20022.js` (JS library), `Repos/iso20022.js` . Source for the practical-use subset (7 families, 2 business areas, ~12% of the standard's breadth; `packs.002/packs.008` not implemented). Detailed analysis: `Research/2026-08-05-iso20022js-praxis-subset.md` .

[BCR-2024-010] Blockchain Commons, BCR-2024-010: XID (eXtensible IDentifier), `Repos/Research/papers/bcr-2024-010-xid.md` . Source for: XID as a 32-byte value, SHA-256 of the inception key, `dereferenceVia` resolution, the 4-byte prefix as a recognition aid only, and machine comparison across all 32 bytes.

[BCR-2026-003] Blockchain Commons, BCR-2026-003: XID Edges, `Repos/Research/papers/bcr-2026-003-xid-edges.md` . Source for: signed edges as verifiable claims.

[FRB] Federal Reserve / FRBServices: Fedwire Funds Service ISO 20022 FAQ. URL: <https://www.frb-services.org/resources/financial-services/wires/faq/iso-20022/overview-implementation-details> (cutover July 14, 2025; RTGS property). Retrieved August 5, 2026.

[TCH] The Clearing House: CHIPS. URL: <https://www.theclearinghouse.org/payment-systems/CHIPS> (migration to ISO 20022 in April 2024). Retrieved August 5, 2026.

[WB-RPW] World Bank: Remittance Prices Worldwide (Q3 2025). URL: <https://remittanceprices.worldbank.org/> (global average 6.36%; low-value P2P remittance benchmark, not a wholesale measure). Retrieved August 5, 2026.

[SWIFT-Reservation] SWIFT CBPR+ go-live (March 20, 2023) and MT retirement (November 2026): common industry knowledge, *not* verified in this research against a reachable primary source (swift.com/iso20022.org not reachable via automated retrieval). Held open for manual re-verification.

text.